UNIVERSIDAD EL BOSQUE

ASIGNATURA

INGENIERIA DE SOFTWARE 2

DOCENTE

ANDRES FELIPE REY CHAVES

INTEGRANTES

JUAN DIEGO MORENO ALAYON

MARIANA LANDINEZ RESTREPO

JEISON MIGUEL GOMEZ GOMEZ

JOHANN SEBASTIAN ORJUELA HEREDIA

Contenido

Objetivo 3
Escenarios de calidad 3
Log de decisiones de Arquitectura 6
Patrones seleccionados 6
1. Factory (Creación del envío base por zona y peso) 6
2. Decorator (Extras de envío) 6
3. Strategy (Promociones y descuentos- métodos de pago) 6
Respuesta a los atributos de calidad identificados 7
Modificabilidad 7
Eficiencia de Desempeño 7
Capacidad. 8
Seguridad JWT. 8
Seguridad API KEYS 8
Demostración de la funcionalidad del aplicativo 9
Inicio de sesión 9
Crear una nueva cuenta 10
Catálogo 11
Resumen del carrito 12
Pedido 13
Pago con tarjeta de calidad 16
Pago Contra Entrega 16
Pago Efecty 17
Factura 18
Asistencia de IA 18
1. Rate Limit 18
2. JWT y API Key 20
3. Número de productos 21
4. Script base de datos 22
Referencias 24

Objetivo

Este taller tiene como objetivo reforzar los conceptos adquiridos a lo largo de la asignatura, como la identificación de requerimientos funcionales y escenarios de calidad, así como aplicar y comprender el uso de patrones de diseño en el desarrollo de soluciones de software. Además, pretende garantizar la protección del API mediante mecanismos de autenticación y control de acceso como JWT y API-KEY.

Escenarios de calidad

Información recibida: el código debe permitir intercambiar o agregar nuevas reglas sin tocar el resto del sistema y debe ser fácil agregar nuevas reglas en el futuro sin que esto implique cambiar todo el sistema o múltiples partes del mismo

	MODIFICABILIDAD
DESCRIPCIÓN	El sistema debe tener la capacidad de intercambiar o agregar nuevas reglas de negocio sin que esto implique cambiar todo el sistema o partes del mismo
STAKEHOLDER	Administrador del sistema, Gestor comercial, Cliente, Equipo de Tecnología
FUENTE	Equipo de Tecnología
ESTIMULO	Se agrega o modifica una de las reglas de negocio del sistema
ARTEFACTO	Sistema
AMBIENTE	Operación normal
RESPUESTA	El sistema se ajusta a los cambios realizados
MEDIDA DE RESPUESTA	El sistema tiene la capacidad de adaptarse a las modificaciones sin introducir defectos o degradar la calidad del sistema

Información recibida: también se debería tener en cuenta un rate limit para que un tercero o un cliente no exceda una cantidad definida de request en un tiempo definido.

	CAPACIDAD
DESCRIPCIÓN	El sistema debe permitir como máximo 4000 solicitudes por minuto de usuarios o terceros
STAKEHOLDER	Administrador del sistema, Gestor comercial, Cliente, sistemas externos
FUENTE	Usuarios (Cliente, sistemas externos)
ESTIMULO	Un usuario realiza solicitudes consecutivas en periodo corto de tiempo
ARTEFACTO	Sistema
AMBIENTE	Periodo de Alta carga
RESPUESTA	El sistema limita la cantidad de solicitudes permitidas al usuario y le muestra error cuando excede el límite
MEDIDA DE RESPUESTA	El sistema limita a 4000 el máximo de solicitudes permitidas por el usuario

Información recibida: En horas pico (18:00–20:00) la app debe responder a consultas y solicitudes en menos de 200ms.

	EFICIENCIA DE DESEMPEÑO
DESCRIPCIÓN	El sistema debe tener la capacidad de responder a consultas y solicitudes en un tiempo menor a 200 ms
STAKEHOLDER	Cliente, administrador del sistema, gestor comercial
FUENTE	Cliente
ESTIMULO	Realiza una consulta o solicitud en el sistema
ARTEFACTO	Sistema QuickCourier
AMBIENTE	Periodo de alta carga
RESPUESTA	El sistema responde a cada una de las solicitudes y consultas
MEDIDA DE RESPUESTA	El sistema tiene la capacidad de responder a consultas y solicitudes en un tiempo menor a 200 ms

Información recibida:

Se requiere una API protegida, de tal forma que cualquier cliente que quiera hacer uso del API deba autenticarse con JWT (Bearer) o incluso un API Key si QuickCourier quisiera integrarse con otras plataformas o que terceros puedan consumir sus servicios.

	SEGURIDAD
DESCRIPCIÓN	El sistema debe asegurar la autenticación de los usuarios mediante JWT para evitar que usuarios no registrados accedan al sistema
STAKEHOLDER	Cliente, administrador del sistema
FUENTE	Cliente
ESTIMULO	El cliente intenta ingresar al sistema sin las credenciales adecuadas
ARTEFACTO	Módulo de autenticación
AMBIENTE	Operación normal
RESPUESTA	El sistema rechaza la solicitud de acceso, devuelve los códigos de error y registrando la información de los logs
MEDIDA DE RESPUESTA	El sistema tiene la capacidad de detener accesos no autorizados mayor al 95%, con un tiempo de validación de < 20ms.

	SEGURIDAD
DESCRIPCIÓN	El sistema debe tener la capacidad de integrarse con otras plataformas o permitir que terceros consuman sus servicios pasando por un proceso de autenticación por medio de API Key
STAKEHOLDER	Administrador del sistema, Sistema externo
FUENTE	Sistema externo
ESTIMULO	Un sistema externo o plataforma quiere conectarse con el sistema
ARTEFACTO	Módulo de autenticación
AMBIENTE	Operación normal
RESPUESTA	El sistema verifica las credenciales del usuario, de ser validas permite el acceso al sistema
MEDIDA DE RESPUESTA	El sistema tiene la capacidad de validar las credenciales en un periodo < 20ms, con una detección de accesos no autorizados >95% y brindando acceso al sistema en un tiempo menor a 100ms

Log de decisiones de Arquitectura

Patrones seleccionados

1. Factory (Creación del envío base por zona y peso)

Se decidió implementar el patrón Factory para la creación de objetos de envío según las reglas comerciales proporcionadas (zona y peso). Esta decisión obedece a que este patrón permite aislar la lógica de construcción de los objetos y delegar la decisión de qué tipo de envío crear a una subclase, cumpliendo de esta manera con el requerimiento de poder agregar nuevas reglas sin tener que afectar el resto del sistema.

2. Decorator (Extras de envío)

En este caso, cada extra (empaque de regalo, manejo frágil, entrega express, envío seguro) se implementa como un decorador que permite agregar comportamiento adicional al objeto de envío base de forma dinámica; permitiendo así la combinación de múltiples extras según el cliente lo requiera sin incurrir en cambios en su estructura base, ni en la creación de muchas clases.

3. Strategy (Promociones y descuentos- métodos de pago)

Teniendo en cuenta que el patrón Strategy se aplica para definir y seleccionar de forma dinámica diferentes estrategias de cálculo o comportamientos, se implementó dicho patrón para los descuentos y los métodos de pago.

En el caso de las promociones, cada tipo de promoción (promoción de fin de semana y primera compra) se establece como una estrategia diferente de cálculo del total del envío, de forma tal que, pueda ser intercambiable en tiempo de ejecución de acuerdo con las reglas que se establezcan en el área comercial. Y en cuanto a los métodos de pago, cada método (Tarjeta, contra entrega, Efecty) funciona como una estrategia que implementa sus propias reglas de validación y lógica de cobro.

Respuesta a los atributos de calidad identificados

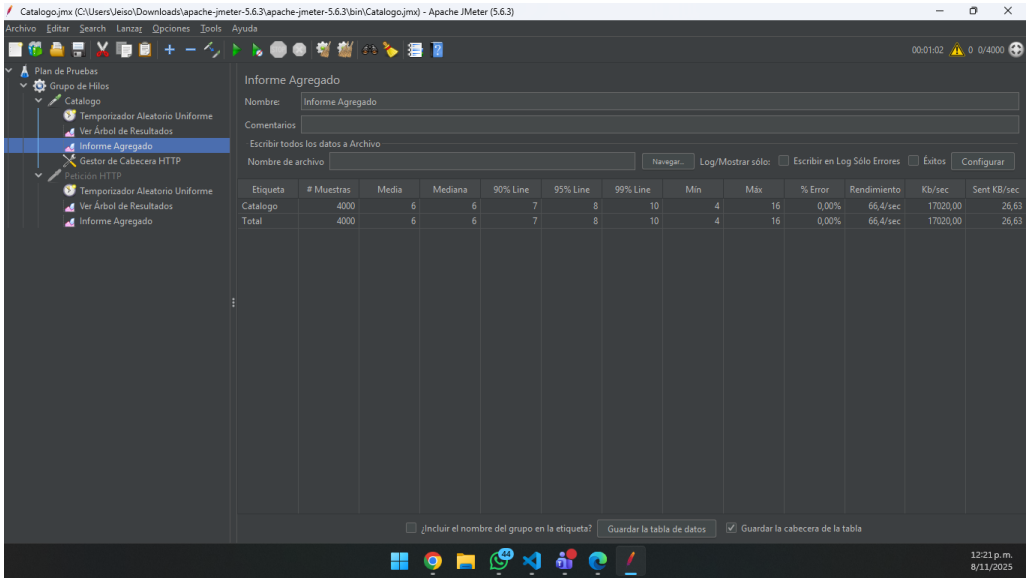
Modificabilidad

Como se mencionó anteriormente, en aras de cumplir con las necesidades del área comercial para la adición y modificación de las reglas de negocio se implementaron los patrones de diseño Strategy, Factory y Decorator que aseguran la modificabilidad del sistema sin afectar su rendimiento.

Eficiencia de Desempeño

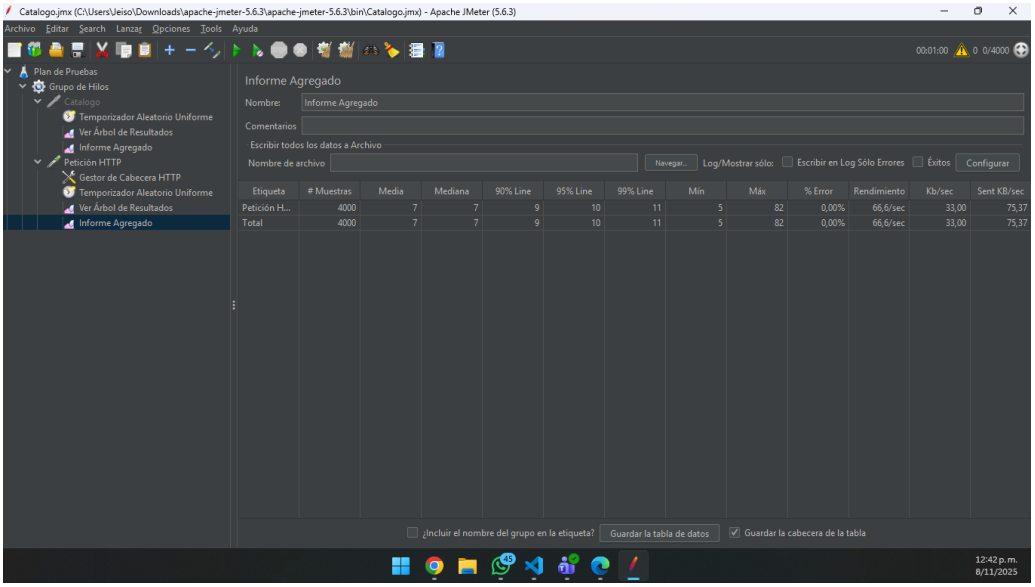
Se realizaron pruebas en los endpoints encargados de manejar consultas y solicitudes, simulando una carga de 4.000 peticiones por minuto. Los resultados muestran que el sistema es capaz de responder a consultas y solicitudes en un tiempo inferior a 200 ms, cumpliendo así con los criterios establecidos en los escenarios de calidad.

Prueba 1



El tiempo máximo registrado para la correcta ejecución de una petición fue de 16 ms, mientras que en el 99 % de los casos el tiempo de respuesta no superó los 10 ms

Prueba 2



El tiempo máximo registrado para la correcta ejecución de una petición fue de 82 ms, mientras que en el 99 % de los casos el tiempo de respuesta no superó los 11 ms

Capacidad.

Se implementó Rate Limit para controlar la cantidad total de peticiones que el sistema puede procesar. Se estableció un límite de 4.000 peticiones por minuto, asegurando así el cumplimiento exitoso de los requerimientos definidos en los escenarios de calidad.

Seguridad JWT.

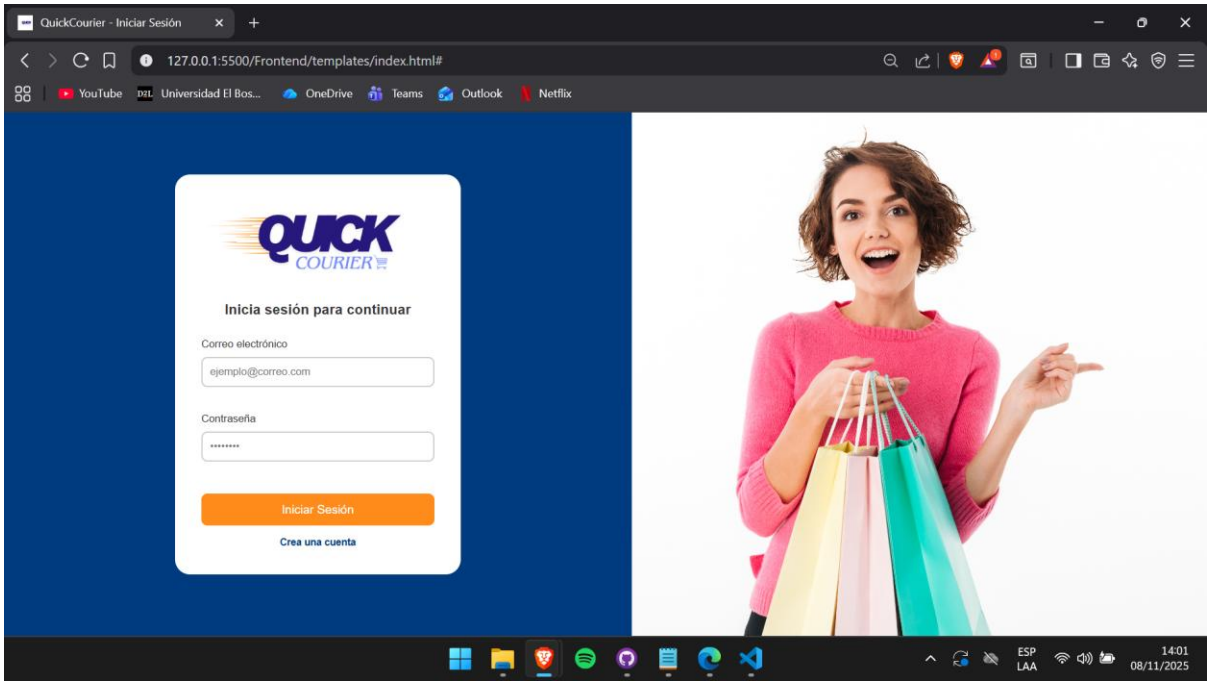
Se implementó un módulo de autenticación basado en JSON Web Tokens (JWT) para garantizar que solo los usuarios autorizados puedan acceder a los endpoints protegidos. Funciona mediante un enfoque sin estado (statless), en el cual el servidor valida cada solicitud mediante un token previamente firmado digitalmente.

Seguridad API KEYS.

Se implemento un módulo que permite a plataformas o servicios de terceros integrase con QuickCourier mediante un esquema de autenticación mediante API KEYS. Este módulo asigna una clave única a cada cliente o aplicación externa, la cual debe incluirse en cada solicitud para validar la autenticación.

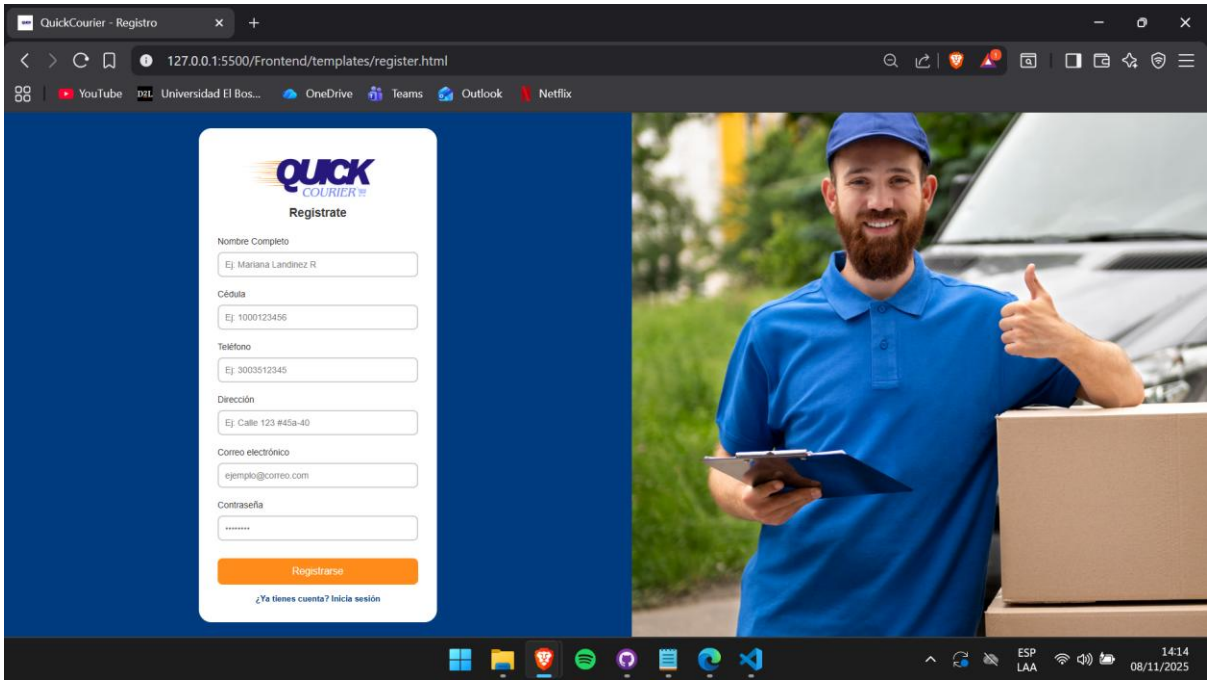
Demostración de la funcionalidad del aplicativo

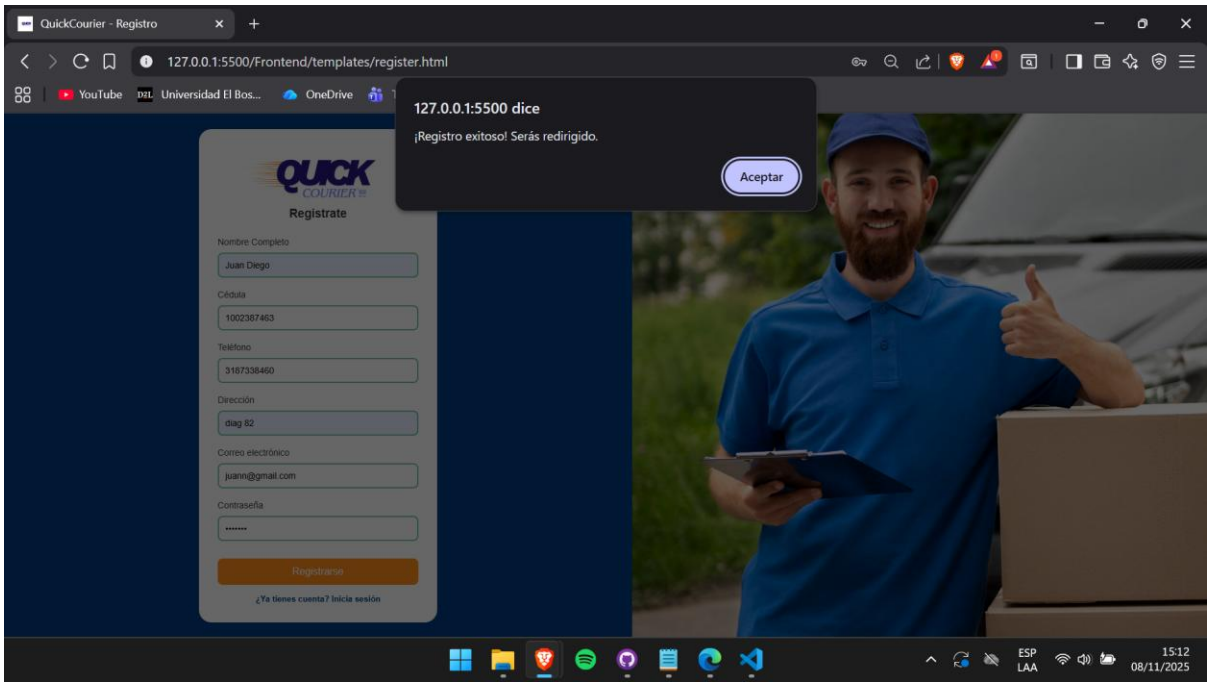
Inicio de sesión



Se presenta una interfaz de inicio de sesión donde el usuario ingresa su correo electrónico y contraseña, al presionar el botón “Iniciar sesión”, las credenciales se validan en el módulo de autenticación y se redirige al usuario al catálogo de la aplicación. Además, cuenta con el botón “Crea una cuenta”, el cual redirecciona al usuario a crear una nueva cuenta en caso de ser necesario.

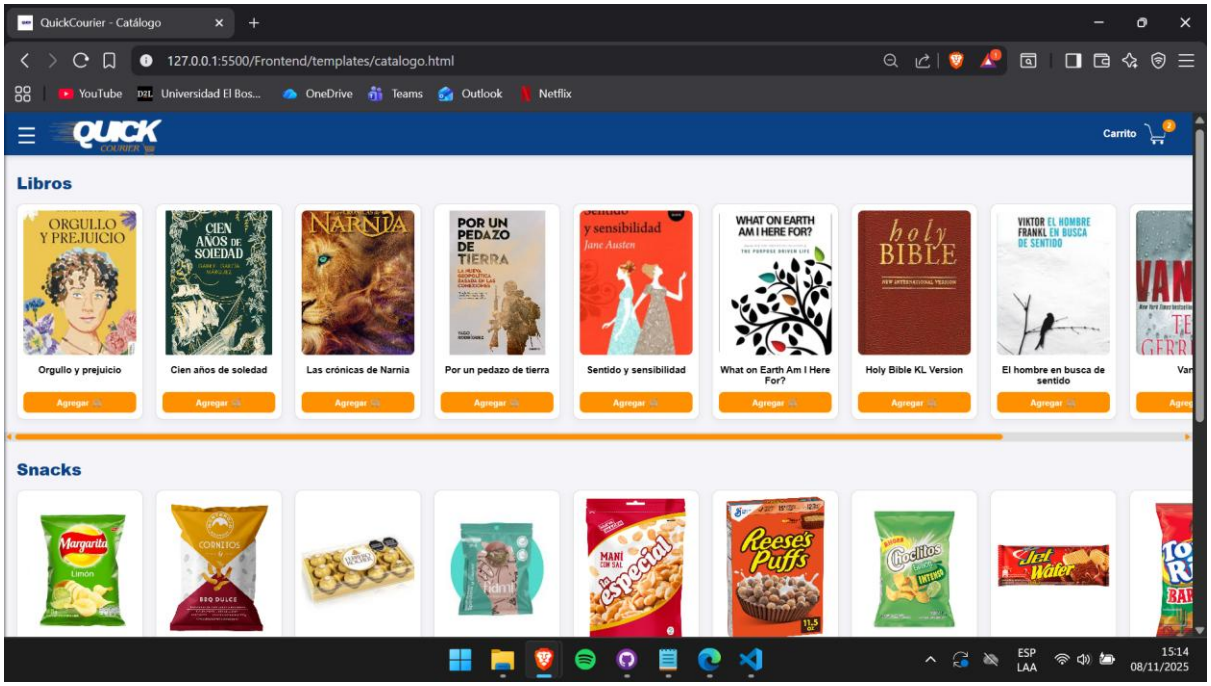
Crear una nueva cuenta





Se presenta una interfaz donde los nuevos usuarios ingresan los datos personales necesarios para la creación de una nueva cuenta en el sistema, al llenar el formulario y presionar el botón “Registrarse”, la información es validada, la contraseña encriptada y guardada en la base de datos del sistema.

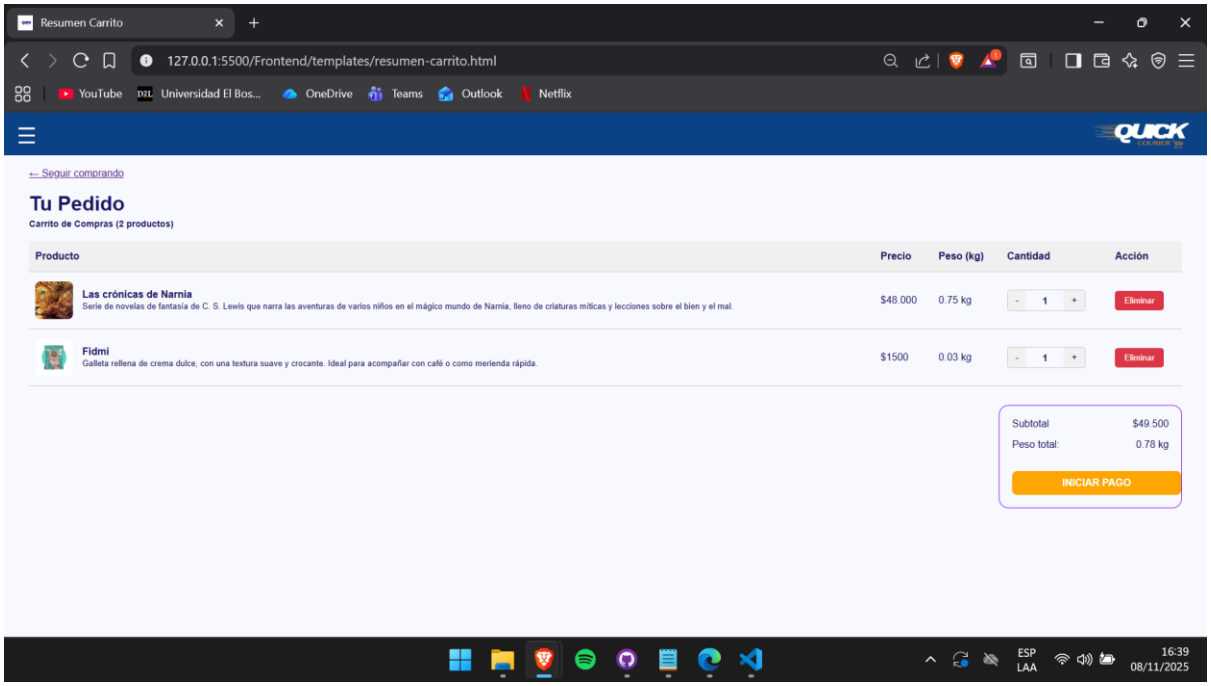
Catálogo



La pantalla de catálogo muestra los productos disponibles dentro del sistema, estos organizados por categorías (libros, snacks y accesorios), cada una con diferentes artículos que el usuario puede agregar al carrito.

Al presionar el botón “Agregar” en cada uno de los productos, estos se guardan en el carrito de compras, posteriormente, el sistema actualiza el ícono del carrito ubicado en la esquina superior derecha mostrando la cantidad de productos agregados. Una vez el usuario haya seleccionado los productos que desea, éste deberá presionar el botón del carrito para ser redirigido al resumen del carrito.

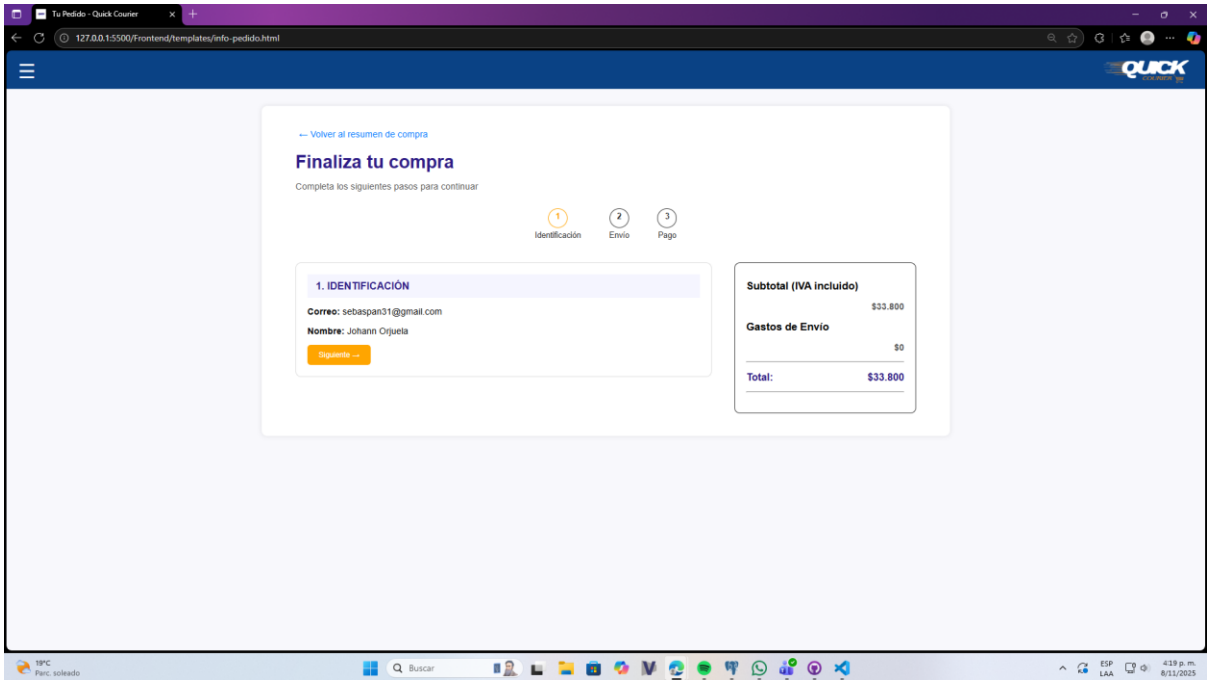
Resumen del carrito



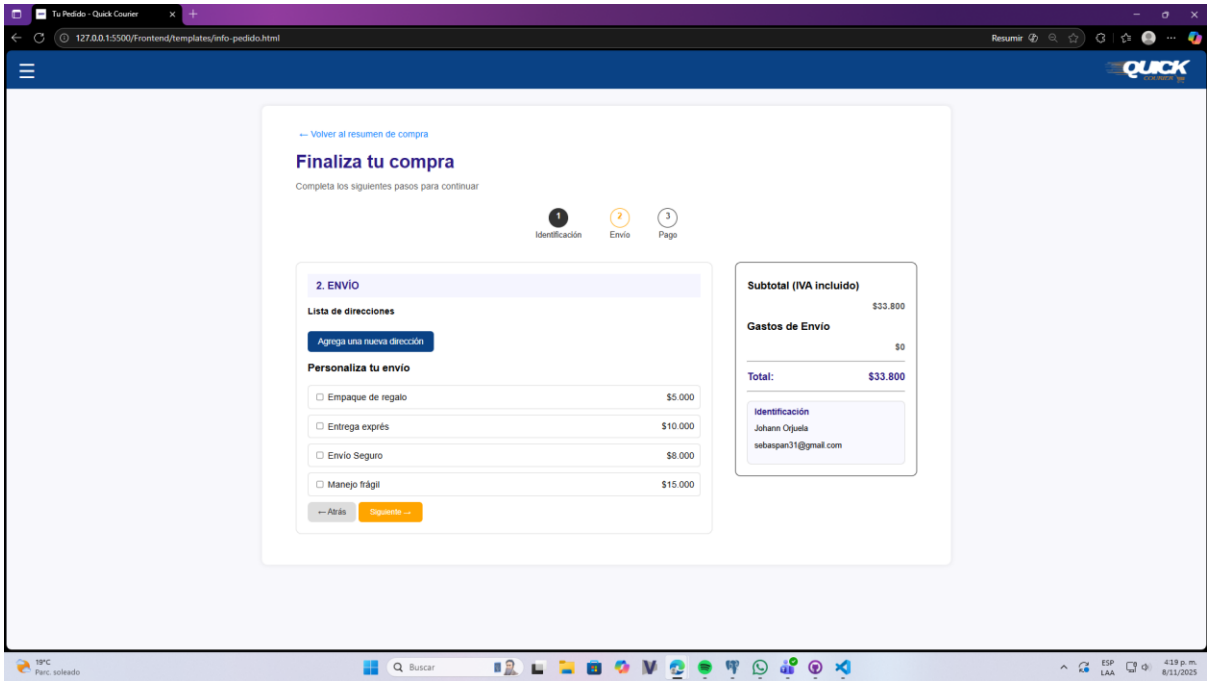
La pantalla de resumen de carrito muestra al usuario los productos seleccionados antes de configurar el envío, su propósito es dar información detallada acerca de cantidades y totales.

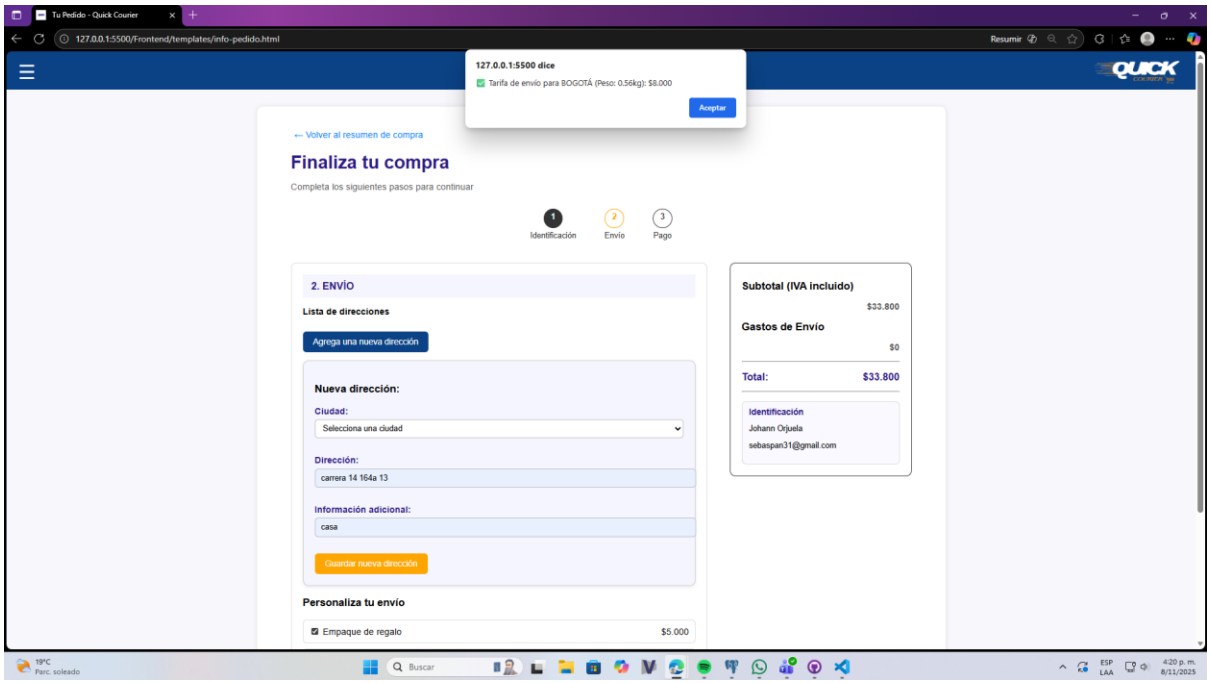
El usuario puede modificar la cantidad de productos en el carrito, tener una descripción detallada de los productos que seleccionó, eliminar elementos seleccionados y ver el subtotal de su compra. Una vez el usuario esté satisfecho debe presionar el botón “Iniciar Pago” para ser redireccionado a la configuración del envió.

Pedido

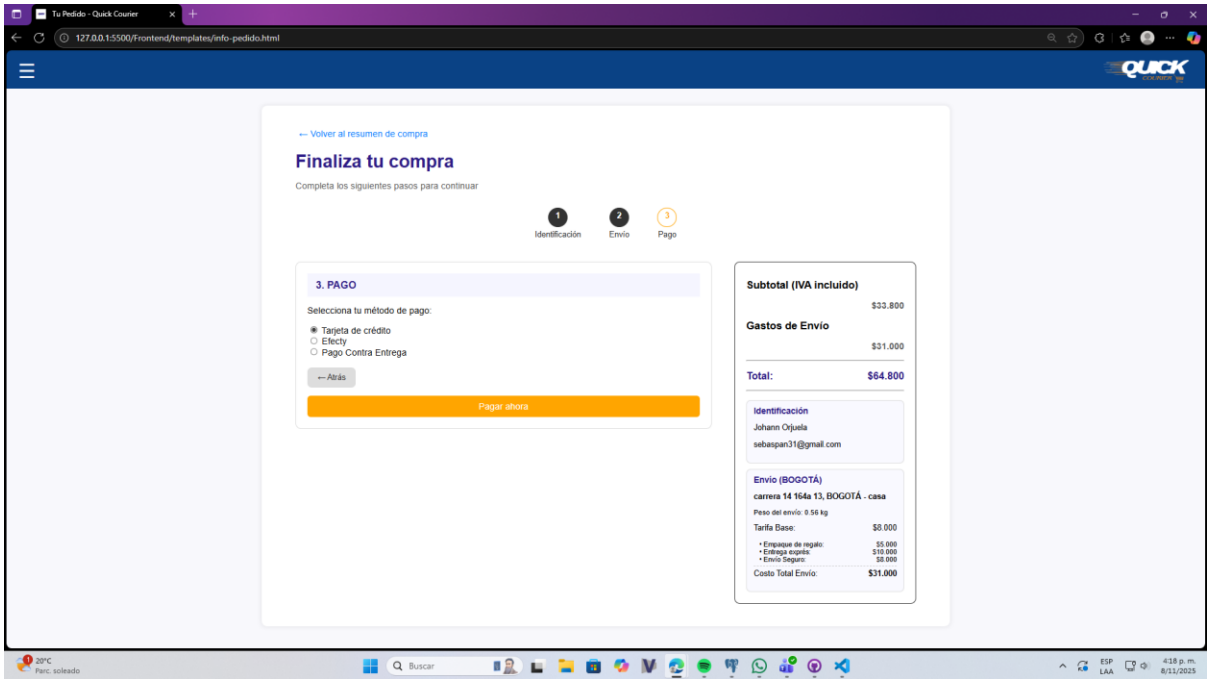


La pantalla de pedido permite al usuario completar un proceso de 3 pasos, Identificación, Envío y Pago. Para esta primera etapa se muestra la información del cliente incluyendo su correo electrónico y nombre registrados en el sistema, a la derecha se encuentra un resumen de cómo va su pedido, que incluye el subtotal, los gastos de envió y el total de la compra. Después de verificar sus datos, cambia a la etapa 2 mediante el botón "**Siguiente**”.





La segunda etapa “**envío**” permite al usuario gestionar la dirección de entrega y adicionar los extras al pedido. Mediante el botón "**Agregar una nueva dirección**” se ingresa la dirección donde se realizará la entrega, para ello, se despliega un formulario donde deberá llenar los datos de ciudad, dirección e Información adicional. Posteriormente, se sumará el costo del envío teniendo en cuenta la ciudad escogida y el peso de los productos; también encontrará los posibles extras que podrá agregar al envío.

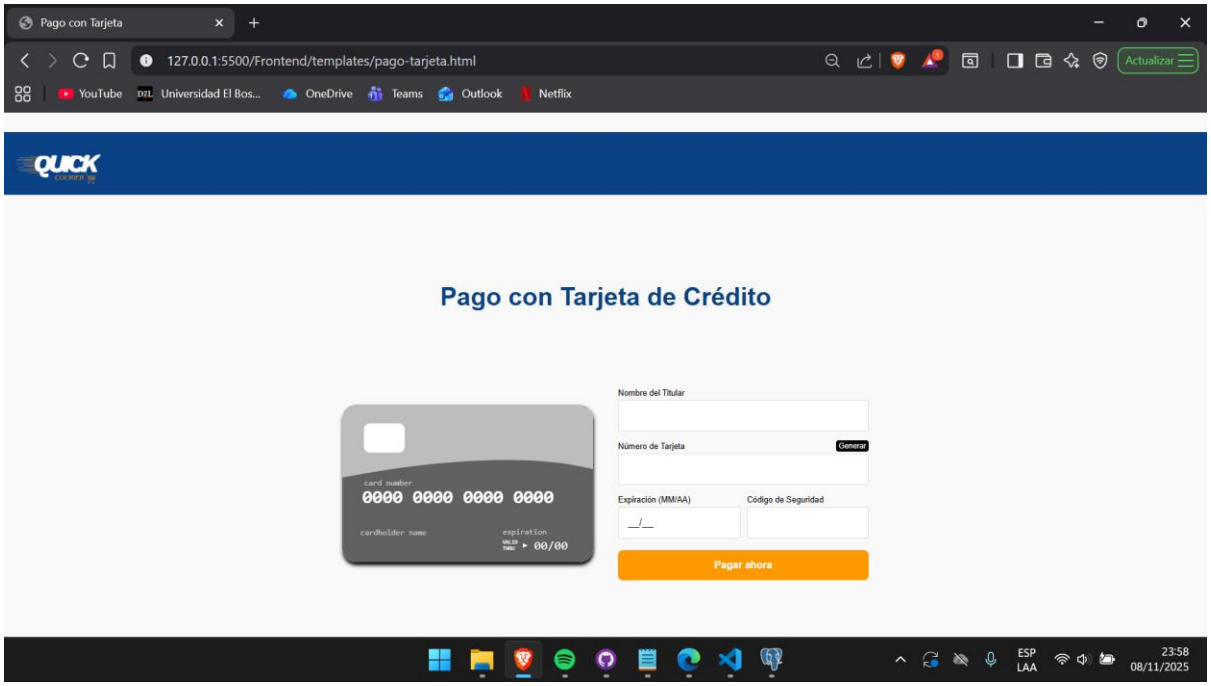


La tercera etapa “**Pago**” permite al usuario seleccionar el medio de pago con el que finalizará su compra, contando con tres opciones disponibles: tarjeta de crédito, Efecty y Pago contra entrega.

En la parte derecha se muestra un nuevo recuadro que contiene el resumen detallado del pedido, incluyendo los datos seleccionados de la etapa anterior.

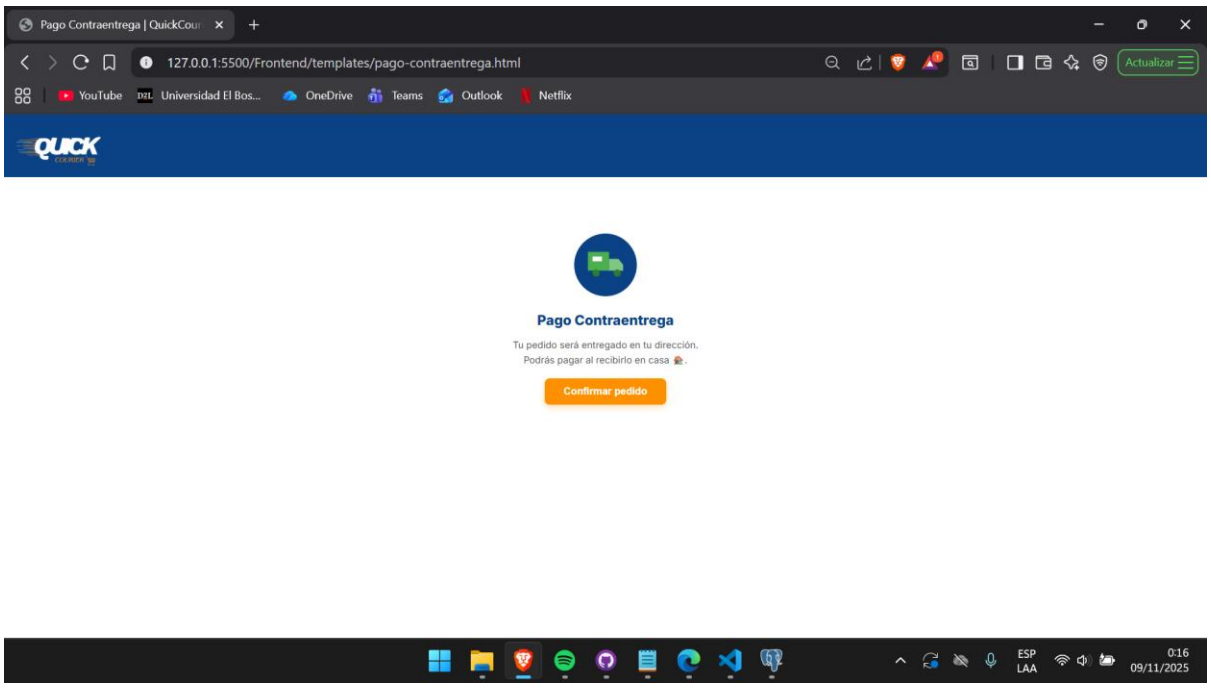
Mediante el botón “**Pagar ahora**”, se confirma la selección y se continúa con la finalización del pedido junto a la generación de su factura.

Pago con tarjeta de calidad

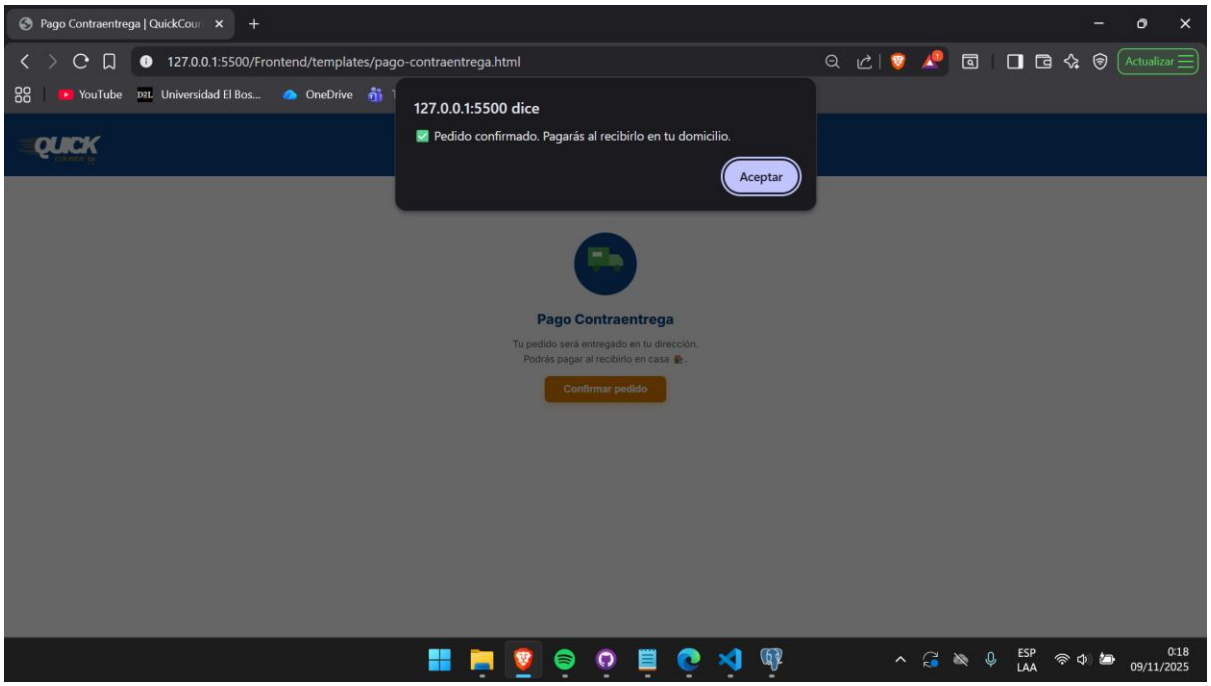


De seleccionar el método de pago “Tarjeta”, en esta pantalla se le permitirá al usuario completar el proceso de pago ingresando la información de su tarjeta de crédito, se solicitan datos (Nombre de titular, Numero de tarjera, Expiración y código de seguridad), el sistema validará que los campos se encuentren completos y que la información contenga el formato adecuado; una vez completados, el usuario deberá presionar el botón “Pagar ahora”, el cual lo redireccionará a la factura generada.

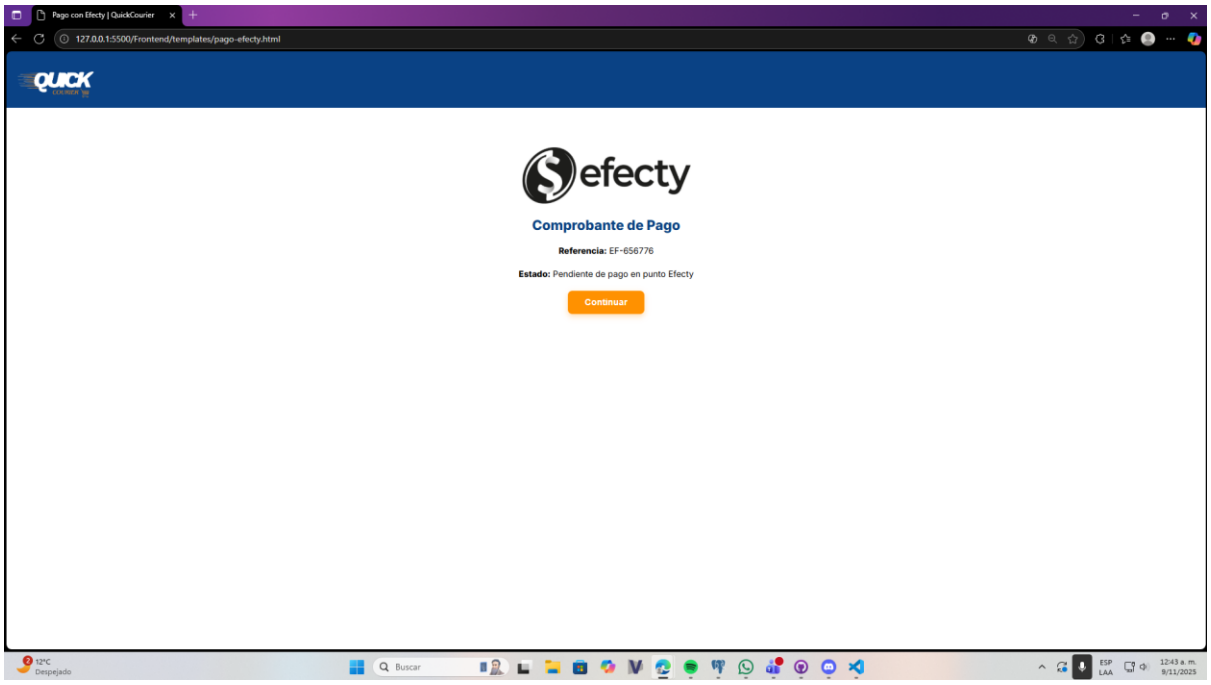
Pago Contra Entrega



De seleccionar el método de pago “Contra Entrega”, En esta pantalla se le informará al usuario que su pedido será enviado a la dirección previamente ingresada. Al presionar el botón “Confirmar pedido”, se redireccionará al usuario a la factura generada.

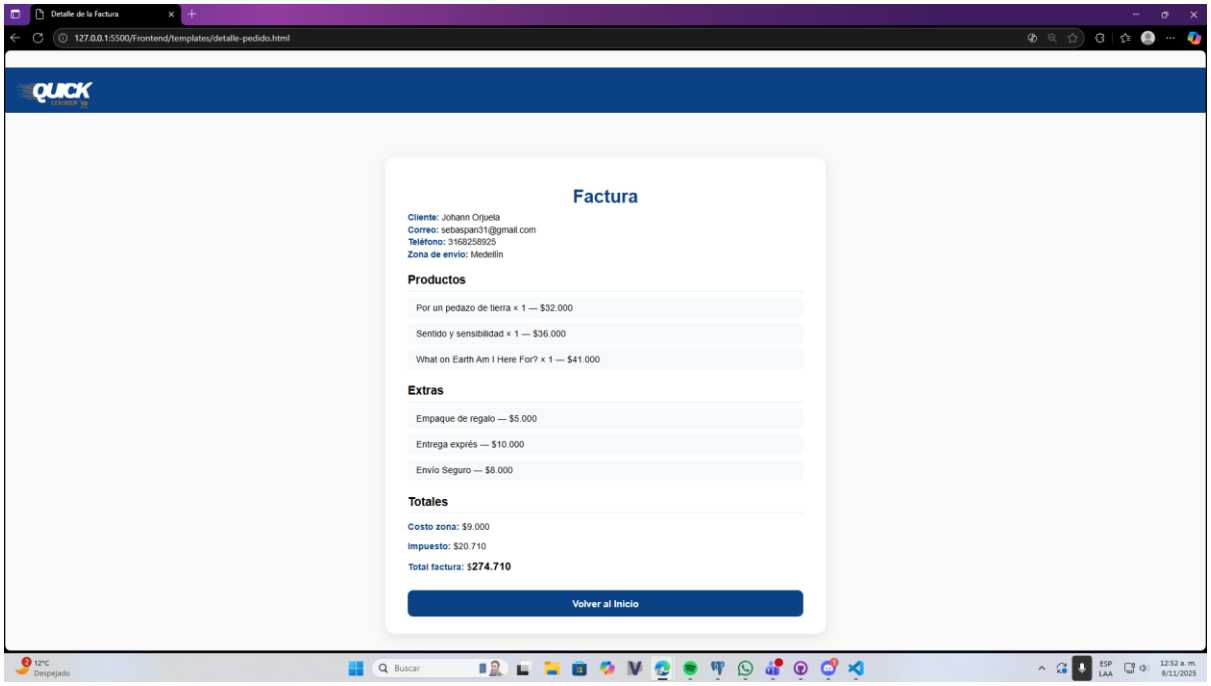


Pago Efecty



De seleccionar el método de pago “Efecty”, en esta pantalla se le permitirá al usuario visualizar el número de referencia del pago, para posteriormente visualizar la factura asociada a su pedido.

Factura



En la pantalla de factura el usuario encontrará el resumen de su pedido, incluyendo: su información personal, el resumen de los productos adquiridos, los diferentes extras seleccionados para su envío y finalmente el monto total de su pedido con las discriminaciones de impuestos a la compra.

Asistencia de IA

1. Rate Limit

Para qué: Comprensión de concepto.

Por qué: Requerimiento del aplicativo.

Modelo: GPT-5, Gemini 2.5 Flash, (Perplexity) Sonar respectivamente

Prompt: “qué es un rate limit y con base en qué se calcula, se conciso”

qué es un rate limit y con base en qué se calcula, se conciso

Un **rate limit** es una restricción que controla cuántas solicitudes o acciones puede realizar un usuario, aplicación o dirección IP en un **periodo de tiempo determinado**.

Se **calcula** con base en:

- **Frecuencia permitida:** número máximo de solicitudes (por ejemplo, 100 por minuto).
- **Ventana de tiempo:** intervalo en que se mide la frecuencia (por ejemplo, 1 minuto, 1 hora).
- **Identificador del cliente:** como el token de API o dirección IP.

👉 En resumen: **Rate limit = (número máximo de solicitudes) / (unidad de tiempo).**

qué es un rate limit y con base en qué se calcula, se conciso

Responder

translate.google
¿Qué es la limitación de velocidad y cómo funciona?

Wallarm
¿Qué es la limitación de velocidad? Significado y...

translate.google
¿Qué es la limitación de velocidad? ¿Qué ocurre si...

MicrosoftLearn
Patrón de limitación de velocidad - Azure...

Pasos del Asistente ›

Un rate limit es un mecanismo que limita la cantidad de solicitudes que un cliente puede hacer a un servidor o API en un período de tiempo específico, normalmente medido en segundos, minutos o horas. Se calcula con base en un número máximo de peticiones permitidas dentro de ese intervalo de tiempo; por ejemplo, permitir 100 solicitudes por minuto. Si se supera ese límite, las solicitudes adicionales se rechazan hasta que inicia el siguiente período de tiempo. El cálculo del rate limit puede ser fijo o dinámico, considerando factores como la carga del servidor o el tipo de usuario. `wallarm +2`

qué es un rate limit y con base en qué se calcula, se conciso

🛡️ ¿Qué es un Rate Limit?

Un **Rate Limit** (límite de frecuencia o tasa) es una estrategia utilizada para **limitar el número de solicitudes** que un usuario, una aplicación o un *bot* puede enviar a una API o a un recurso de red durante un **período de tiempo específico**.

Su objetivo principal es:

- **Proteger** los servidores de un uso excesivo o abusivo (como ataques de Denegación de Servicio o Fuerza Bruta).
- **Asegurar** la disponibilidad y el rendimiento para todos los usuarios, gestionando el flujo de tráfico de red.

⏳ Base de Cálculo

El cálculo del *rate limit* se basa en una combinación de **métricas y una ventana de tiempo**.

Los parámetros comunes para definir y calcular un *rate limit* son:

- **Número de Operaciones / Solicitudes:** Es el **límite máximo** de llamadas o peticiones permitidas.
  - *Ejemplo:* 500 llamadas por minuto.
- **Período de Tiempo (Ventana de Tiempo):** Es el **lapso de tiempo** durante el cual se aplica el límite. Después de este período, el contador de solicitudes se reinicia.
  - *Ejemplo:* Por minuto, por hora, por día.

2. JWT y API Key

Para qué: Comprensión de concepto.

Por qué: Requerimiento del aplicativo.

Modelo: GPT-5, Gemini 2.5 Flash, (Perplexity) Sonar

Prompt: “que es JWT y API Key, se conciso”

que es JWT y API Key, se conciso”

JWT (JSON Web Token): es un token en formato JSON usado para autenticar y autorizar usuarios. Contiene información firmada (como el ID del usuario y roles) que el servidor puede verificar sin consultar la base de datos.

API Key: es una clave única que identifica y autentica a una aplicación o cliente que accede a una API, generalmente usada para controlar el acceso y registrar el uso del servicio.

que es JWT y API Key, se conciso”

Dos herramientas clave para la seguridad en APIs son **JWT** (JSON Web Token) y **API Key**. Ambas se usan para **autenticación** y **autorización**, pero funcionan de manera diferente.

🔑 API Key (Clave de API)

Una **API Key** es una **cadena de texto única y secreta** que se usa para **identificar** a un cliente (una aplicación o desarrollador) que accede a una API.

- **Función principal:** Es similar a una contraseña o credencial estática para la aplicación.
- **Mecanismo:** El cliente envía la clave en cada solicitud HTTP (generalmente en el *header* o como un parámetro de consulta). El servidor la busca en una base de datos para verificar si la clave es válida y qué permisos tiene asociados el cliente.
- **Información:** Por sí misma, **no contiene información** sobre el usuario o los permisos; es solo un identificador.

🛡️ JWT (JSON Web Token)

Un **JWT** es un **token autocontenido** (autónomo) y **firmado digitalmente** que se utiliza para transmitir información de forma segura entre partes.

- **Función principal:** Se utiliza comúnmente en la **autenticación basada en tokens** para probar la identidad del usuario y sus permisos (autorización).

que es JWT y API Key, se conciso”

Responder Imágenes

translate.google
Introducción al token web JSON - jwt.io

OpenWebinars.net
Qué es Json Web Token y cómo funciona

ibm.com
JSON Web Token (JWT)

Wikimedia Foundation,...
JSON Web Token - Wikipedia, la enciclopedia...

Pasos del Asistente ›

JWT (JSON Web Token) es un estándar abierto (RFC 7519) que define un método compacto y autónomo para transmitir información segura entre partes en forma de un objeto JSON firmado digitalmente. Se usa principalmente para autenticar y autorizar usuarios, sin necesidad de mantener sesiones en el servidor, ya que contiene toda la información necesaria sobre la identidad y privilegios del usuario.

API Key es una clave única que se utiliza para identificar y autenticar a un consumidor o cliente que accede a una API. Es un simple token estático que se pasa en las solicitudes para controlar el acceso, pero no contiene información del usuario ni de permisos embebidos, a diferencia del JWT.

En resumen, el JWT es un token seguro y con contenido de identidad y permisos, mientras que la API Key es un identificador estático para controlar acceso a la API. `translate.google` +4

3. Número de productos

Para qué: Entender un escenario real sobre la cantidad de productos manejados por un e-commerce.

Por qué: Requerimiento del aplicativo.

Modelo: GPT-5

Prompt: “Cuantos productos puede tener un ecomerce en su bd?”

cuantos productos puede tener un ecomerce en su bd?

Buena pregunta — y la respuesta depende del **tipo de base de datos**, su **infraestructura**, y **cómo está diseñada**. Pero te explico claramente por niveles 👇

🧩 1. Depende del tipo de base de datos

🔹 **Relacionales (MySQL, PostgreSQL, SQL Server)**

- En teoría, pueden manejar **millones de productos** sin problema si están bien optimizadas.
- Por ejemplo:
  - **MySQL** puede manejar tablas con más de **100 millones de filas**.
  - **PostgreSQL** puede llegar incluso a **billones de registros** con configuraciones empresariales.
- Lo que importa no es el número de filas, sino:
  - Índices bien diseñados (por ejemplo, índices en `id_producto`, `categoria`, `precio`).
  - Consultas eficientes (sin traer todo de golpe).
  - Caché y paginación en las peticiones.

🧠 2. Depende del nivel del e-commerce

Tipo de e-commerce	Ejemplo	Cantidad típica de productos
Pequeña tienda	Tienda local	50 – 1,000 productos
E-commerce mediano	Pyme o minorista online	1,000 – 100,000 productos
Gran marketplace	Amazon, Mercado Libre	>10 millones de productos

4. Script base de datos

Para qué: Poblamiento de la base de datos.

Por qué: Requerimiento del aplicativo.

Modelo: GPT-5

Prompt: “dame un script para crear 5000 productos”

dame un script para crear 5000 productos con base en la siguiente tabla

```
CREATE TABLE producto (
    id_producto SERIAL PRIMARY KEY,
    nombre_prod VARCHAR (255),
    descripcion_prod VARCHAR (255),
    precio_uni_prod DECIMAL (10,2),
    categoria_prod VARCHAR (255),
    peso_prod DECIMAL (10,2),
    ruta_prod VARCHAR (255)

);
```

para el campo de la ruta pon siempre static/img/prod/cien_anos_soledad.jpg

Perfecto 💪
Aquí tienes un **script en PostgreSQL** que genera automáticamente **5000 productos** de prueba para tu tabla `producto`, con datos aleatorios realistas y la ruta fija que pediste.

Copiar código

```python
import random

# Nombre del archivo SQL
output_file = "insert_productos.sql"

# Categorías válidas
categorias = ["Accesorios", "Snacks", "Libros"]

# Palabras base para nombres
nombres_base = [
    "Producto", "Artículo", "Objeto", "Elemento", "Item", "Referencia",
    "Artículo Premium", "Producto Especial", "Artículo Edición Limitada"
]

# Generar archivo con 5000 INSERTs
with open(output_file, "w", encoding="utf-8") as f:
    f.write("INSERT INTO producto (nombre_prod, descripcion_prod, precio_uni_prod, categoria_prod,

    for i in range(5000):
        nombre = f"{random.choice(nombres_base)} {i+1}"
        descripcion = f"Descripción del {nombre.lower()}, útil y de alta calidad."
        precio = round(random.uniform(10000, 150000), 2)
        categoria = random.choice(categorias)
        peso = round(random.uniform(0.1, 2.0), 2)
        ruta = "static/img/prod/cien_anos_soledad.jpg"

        # Agregar coma o punto y coma según la posición
        separador = "," if i < 4999 else ";"

        f.write(f"('{nombre}', '{descripcion}', {precio}, '{categoria}', {peso}, '{ruta}'){separad

print(f"✅ Archivo '{output_file}' generado con 5000 productos correctamente.")
```

Conclusiones

1. Se pudo concluir que la implementación de la seguridad a la API resultó bastante compleja pues se desconocía completamente la librería y el funcionamiento de los tokens.
2. El uso de Rate Limit resulta muy efectivo pues limita las peticiones al sistema, evitando así, la posible saturación del servidor y permitiendo asegurar los requerimientos de calidad asociados a la capacidad de respuesta del sistema.
3. La identificación y selección de los patrones de diseño debe responder adecuadamente a las necesidades planteadas por el cliente, no se debe implementar patrones de forma deliberada, ya que de lo contrario se puede incurrir en un incremento innecesario de la complejidad del código.

Referencias

[1] R. Programando en JAVA, “JWT SPRING SECURITY - SPRING BOOT 3,” *YouTube*, Aug. 2023. [Online]. Available: https://www.youtube.com/watch?v=-Z4a0bKr2Pg